Intelligent Forest Advisory & Multi-Structure Decision System (IFAMDS)

1 Implementation Requirements

Students must work in group of two and implement a fully working C++ console-based simulation system with a menu-driven interface. The system must demonstrate correct usage of all required data structures through at least five forest-based scenarios.

- C++ console-based working simulation system
- Menu-driven interface covering all major operations
- Integration of required data structures (arrays, linked lists, stacks/queues, trees, graphs, hash tables)
- Execution of all **five complete scenarios**
- Console output must clearly show system state changes and processing flow
- Proper function-level comments throughout the code
- Key algorithms (BFS, DFS, hashing, scheduling) must be commented
- Time complexity must be mentioned for major operations (arrays, linked lists, queues, trees, graphs, hashing)
- Modular, readable, and well-structured code design
- Proper naming conventions and indentation
- **Short Report (2–4 pages):** Include only:
  - System overview
  - Data structure usage summary

  - Brief explanation of 5 scenarios
  - Screenshots of outputs and menu execution

- **Submission:**
  - Source code (.cpp files)
  - Short report (2–4 pages)

2 Introduction

The Intelligent Forest Advisory & Multi-Structure Decision System (IFAMDS) is a large-scale computational system which models a real-world forest management environment where data flows through multiple processing layers and decisions emerge from interactions between structures rather than isolated computations.

The design emphasizes complexity, interdependency, and algorithmic depth by ensuring that each data structure is not only implemented but actively contributes to system-wide processing.

2.1 Array-Based Environmental Processing Layer

2.1.1 Structural Instances

- A1: Static Environmental Baseline Array
- A2: Dynamic Sensor Stream Array
- A3: Static Forest Grid Matrix
- A4: Dynamic Terrain Expansion Matrix

2.1.2 Functional Role

These arrays are used to store and process forest data such as temperature, humidity, smoke level, and fire signals collected from different forest zones.

- **Static arrays:** These store fixed reference values of normal forest conditions. These values do not change during execution. They are used for comparison with live sensor data.
  Meaning of Static: fixed / constant values that represent normal environment (baseline reference system).
  Example: Normal temperature = $25°C$, humidity = 60%, smoke level = 0.

- **Dynamic arrays:** These store sensor values that change during program execution. Every new sensor reading is added or updated in the array.
  Meaning of Dynamic: continuously changing data during runtime (live system updates).
  Example: Temperature over time = $[25, 27, 30, 33]$.

- **1D arrays (Sequential data):** These store readings in time order for one sensor or one zone. This helps to track how values change over time.
  Meaning of Sequential: arranged in order (time-based flow of data).
  Example:
  $$[t_1 = 25,\ t_2 = 26,\ t_3 = 28]$$
  Purpose: used to detect trends such as increasing temperature or sudden drops.

- **2D arrays (Spatial grid):** These store forest zones in a matrix form (rows and columns). Each cell represents one forest area.
  Meaning of Spatial Grid: a map-like structure where each position represents a physical location.
  Example:
  $$\begin{bmatrix} 25 & 30 & 28 \\ 27 & 45 & 32 \\ 26 & 29 & 31 \end{bmatrix}$$
  Meaning of each cell: environmental value of a specific forest zone.

2.1.3 Operations

- **Environmental data ingestion:** This means collecting raw data from sensors and storing it in arrays.
  Meaning of ingestion: receiving external sensor signals and storing them into system memory.
  Example: A sensor sends $40°C$ and it is stored in array A2.

- **Spatial interpolation:** This means estimating a missing value using nearby values. It is used when a sensor fails.
  Meaning of interpolation: calculating unknown value using surrounding known values.
  Formula:
  $$value = \frac{top + bottom + left + right}{4}$$
  Example: If one zone is missing, its value is calculated using surrounding zones.

- **Boundary detection:** This means finding sudden changes between two nearby zones. It helps detect fire spread or abnormal conditions.
  Meaning of boundary: a region where values change sharply between neighbors.
  Example: One zone = $25°C$, next zone = $50°C \rightarrow$ boundary detected.

- **Threshold-based anomaly filtering:** This means checking if values go beyond safe limits. If yes, it is marked as an anomaly (danger condition).
  Meaning of threshold: a predefined safe limit.
  Meaning of anomaly: any value that deviates strongly from normal behavior (possible fire, error, or abnormal event).

  Rules:

  - Temperature $> 45°C \rightarrow$ fire risk
  - Smoke level $> 70 \rightarrow$ possible fire
  - Humidity $< 20\% \rightarrow$ dry condition

  Any value outside safe range is treated as an **anomaly**.

2.2 Linked Data Event Memory Layer

This layer stores forest events in a connected sequence where each event is linked to the next event. It is used to record how forest conditions change over time instead of storing separate values. The structure allows tracking of past, present, and future event flow in a continuous manner.

The system treats each event as a time-stamped record:

$$Event_i = (value_i, time_i, zone_i)$$

2.2.1 Structural Instances

- **L1–L3: Singly Linked Event Streams** In this structure, each event is connected only to the next event in one direction. It is used to store events in time order. Example: $\text{Event}_1 \rightarrow \text{Event}_2 \rightarrow \text{Event}_3$

  Traversal rule:

  $$Event_i \rightarrow Event_{i+1}$$

  Explanation: This means system can only move forward in time sequence.

- **L4–L6: Doubly Linked Correction Chains** In this structure, each event is connected in both forward and backward direction. It allows moving in both directions for correcting wrong or updated data. Example: $\text{Event}_1 \leftrightarrow \text{Event}_2 \leftrightarrow \text{Event}_3$

  Navigation rule:

  $$Event_i \leftrightarrow Event_{i-1}, Event_{i+1}$$

  Explanation: This means system can move backward to fix old data or forward to update future data.

- **L7–L10: Circular Monitoring Loops** In this structure, the last event is connected back to the first event forming a loop. It is used for continuous monitoring where system never stops updating. Example: $\text{Event}_1 \rightarrow \text{Event}_2 \rightarrow \text{Event}_3 \rightarrow \text{Event}_1$

  Loop rule:

  $$Event_n \rightarrow Event_1$$

  Explanation: This means monitoring never stops and repeats continuously.

2.2.2 Functional Role

- **L1: Raw Event Stream** Stores direct sensor readings without processing or filtering. Example: $[25°C, 28°C, 30°C]$

  Storage rule:

  $$Data_{stored} = Data_{sensor}$$

  Explanation: Values are stored exactly as received from sensors without modification.

- **L2: Verified Event Stream** Stores sensor readings after removing noise or incorrect values.

  Noise detection (simple rule): Noise is detected when a value changes too quickly compared to previous value.

  $$|Value_i - Value_{i-1}| < \delta \Rightarrow Normal$$

  If:

  $$|Value_i - Value_{i-1}| \geq \delta \Rightarrow Noise$$

  Incorrect value detection: If value is outside physical limits:

  $$Value < 0 \Rightarrow Invalid \quad \text{or} \quad Value > MaxLimit \Rightarrow Invalid$$

Filtering rule:

$$Data_{verified} = Data_{raw} - Noise$$

- **L3: Anomaly Event Stream** Stores abnormal or unexpected readings that do not match normal forest behavior. Example: sudden jump from $25°C$ to $60°C$

  Anomaly condition:

  $$|Value_{current} - Value_{normal}| > \theta$$

  where θ is safe threshold.

  Simple meaning: If value is valid but too different from normal pattern, it becomes anomaly.

  Example of anomaly: If normal forest temperature is $25°C$ but suddenly becomes $60°C$, it is marked as anomaly because it may indicate fire or sensor fault.

- **L4: Forward Correction Chain** Used to fix future events when a past event is corrected.

  Update rule:

  $$Event_{i+1} = f(Event_i^{corrected})$$

  OR

  $$next\ value = updated\ previous\ value$$

- **L5: Backward Correction Chain** Used to go back and correct previous events when a new correct value is found.

  Backtracking rule:

  $$Event_{i-1} = f(Event_i^{new})$$

  OR

  $$previous\ value = corrected\ value$$

  Explanation: System updates previous data if new information is more accurate.

- **L6: State Synchronization Chain** Ensures all corrected and updated events remain consistent across the system.

  Consistency rule:

  $$Event_i^{all\ nodes} = Event_i^{global}$$

  OR

  $$all\ systems = same\ value$$

Explanation: Every module sees same updated value.

- **L7: Local Monitoring Loop** Continuously monitors a single forest region in a repeating cycle.

  Cycle rule:

  $$Monitor(zone) \rightarrow repeat(t)$$

  OR

  $$repeat\ monitoring(zone)$$

  Explanation: One zone is checked repeatedly over time.

- **L8: System-wide Monitoring Loop** Continuously monitors the entire forest system in a repeated cycle.

  Scan rule:

  $$\sum_{i=1}^{n} Zone_i$$

  OR

  $$check\ all\ zones$$

  Explanation: All zones are checked one after another.

- **L9: Emergency Monitoring Loop** Activated when dangerous conditions are detected such as fire, smoke, or extreme temperature.

  Trigger condition:

  $$Value > Threshold \Rightarrow Emergency$$

  where:

  $$Threshold = safe\ limit$$

  Explanation: System switches to fast response mode when danger appears.

- **L10: Stability Monitoring Loop** Monitors long-term normal forest conditions to ensure the environment remains stable over time.

  Stability condition:

  $$|Value_t - Value_{t-1}| < \epsilon$$

  OR Rule:

  $$change < small\ limit \Rightarrow stable$$

  Explanation: Small changes mean stable environment.

2.3 Queue-Based Scheduling Engine

This layer is used to manage the order in which tasks are processed. A queue follows the principle of **First In First Out (FIFO)**, meaning the first task added is the first one executed. This is important in forest systems where different types of tasks arrive continuously such as sensor updates, alerts, and emergency signals.

2.3.1 System Justification

Multiple queues are used because not all tasks have the same importance or processing requirement. Separating them helps the system handle normal monitoring and emergency situations without confusion or delay.

- Routine tasks are processed in normal order to maintain continuous monitoring.
- Emergency tasks are handled separately so they do not wait behind normal tasks.
- Mixed decision tasks are processed after combining multiple inputs.

2.3.2 Queue Instance Mapping and Description

Q1: Routine Monitoring Queue

This queue stores normal sensor updates that are not urgent.

Example: Sensor sends readings every minute:

$$(25°C, 26°C, 27°C)$$

These are processed one by one in order of arrival.

Q2: Continuous Surveillance Queue

This queue stores frequent updates from sensitive forest zones where conditions change quickly.

Example: In a dry zone, smoke readings arrive frequently:

$$(10, 12, 15, 18, 25)$$

Each new reading is added at the end and processed in sequence.

Q3: Emergency Response Queue

This queue stores urgent events that indicate danger such as fire or extreme temperature.

Example: If temperature suddenly jumps:

$$25°C \rightarrow 60°C$$

or smoke level rises from 5 to 90, the event is added here immediately for fast processing.

Q4: Multi-Factor Decision Queue

This queue stores tasks where multiple inputs must be combined before making a final decision.

Example: A fire decision depends on:

- Temperature = 45°C
- Smoke = 70
- Wind speed = high

These values are combined and then processed together to decide whether evacuation is needed.

2.3.3 Operations

- Enqueue: Adding a new task when a sensor sends data. Example: new smoke reading enters Q3.
- Dequeue: Removing the oldest task after it is processed. Example: first temperature reading is processed and removed.
- Priority switching: Moving emergency tasks ahead of normal ones. Example: fire alert moves before routine temperature updates.
- Load balancing: Distributing tasks so one queue does not become too large. Example: splitting heavy sensor traffic across Q1 and Q2.

2.4 Tree-Based Decision Intelligence Layer

This layer is used for making structured decisions in the forest system. A tree is used because it allows decisions to move step-by-step from general information (forest level) to specific actions (zone level).

Each decision is not random; it is based on conditions such as fire intensity, resource availability, and risk level.

2.4.1 System Justification

Twelve trees are used because forest decision-making requires separating different types of logic such as location, resources, incidents, and final actions.

A basic decision rule used in this system is:

$$Decision\ Score = w_1(Fire) + w_2(Smoke) + w_3(Temperature)$$

where:

- w_1, w_2, w_3 are importance weights
- Fire, Smoke, Temperature are sensor inputs

If Decision Score > threshold, emergency action is triggered.
Example: If:

$$Fire = 0.8, \quad Smoke = 0.7, \quad Temperature = 0.9$$

then:

$$Score = 0.4(0.8) + 0.3(0.7) + 0.3(0.9) = 0.79$$

If threshold = 0.6 → emergency is activated.

2.4.2 Tree Instance Mapping and Explanation

Structural Trees (Forest Division Logic)

- **T1: Zone Hierarchy Tree** Divides forest into structured zones. Example: Forest → Zone A → Zone A1 → Zone A1-1
- **T2: Sub-Zone Decomposition Tree** Splits zones into smaller monitoring units. Example: Each zone is divided into 4 directions (N, S, E, W)
- **T3: Terrain Classification Tree** Classifies land based on risk level.

Terrain risk score:

$$Terrain\ Risk = \frac{Slope + Dryness + Density}{3}$$

Example:

$$Slope = 0.8,\ Dryness = 0.9,\ Density = 0.7 \Rightarrow Risk = 0.8$$

Resource Trees (Availability of Control Resources)

- **T4: Water Resource Tree** Tracks available water sources.

  Availability formula:

$$Water\ Availability = \frac{Available\ Water}{Required\ Water}$$

Example: If available = 80L, required = 100L:

$$0.8 \Rightarrow sufficientbutlimited$$

- **T5: Fire Control Resource Tree** Tracks fire response tools.
- **T6: Equipment Allocation Tree** Assigns resources to zones based on priority:

$$Priority = Risk \times Impact$$

Example: Zone A risk = 0.9, impact = 0.8

$$Priority = 0.72$$

Incident Trees (Event Classification)

- **T7: Fire Classification Tree** Fire level is calculated using:

$$Fire\ Level = \alpha(Temperature) + \beta(Smoke)$$

Example: High temperature + high smoke → major fire alert

- **T8: Wildlife Activity Tree** Detects movement patterns.
- **T9: Human Activity Tree** Detects human presence or intrusion.

Human risk score:

$$Human\ Risk = Movement \times Restricted\ Area\ Factor$$

Decision Trees (Final Actions)

- **T10: Local Decision Tree** Makes decisions for one zone.

  Rule:

$$If\ Risk > 0.6 \Rightarrow Activate\ Local\ Response$$

- **T11: Regional Escalation Tree** Spreads alert to nearby zones.

  Spread condition:

$$If\ Fire\ Spread\ Rate > 0.5 \Rightarrow Escalate$$

- **T12: Global Emergency Tree** Full system emergency control.

  Global condition:

$$If \sum Risk_{zones} > Threshold \Rightarrow Global\ Alert$$

2.5 Graph-Based Routing Layer

This layer is used to represent the forest as a set of connected zones. Each zone is treated as a point (node), and connections between zones are called paths (edges). These connections help the system understand how fire, movement, or resources can travel across the forest.

Example: If Zone A is connected to Zone B, then fire or rescue teams can move from A to B through that path.

2.5.1 Structural Instances

- **G1: Adjacency List Graph** Stores only direct connections of each zone. Example: Zone A → Zone B, Zone C
- **G2: Adjacency Matrix Graph** Stores connection information in table form. Example: If Zone A is connected to Zone B, value = 1, otherwise 0.

2.5.2 Functional Role

- G1 is used when each forest zone has only a few nearby connections. Example: Mountain forest where paths are limited.
- G2 is used when every zone is checked against all other zones in a grid format. Example: Dense forest monitoring system.

2.5.3 Operations

1. BFS (Breadth First Search)

Used to check all nearby zones step by step.

Example: If fire starts in Zone A:

$$Zone\ A \rightarrow Zone\ B \rightarrow Zone\ C$$

This helps the system predict how fire spreads level by level.

2. DFS (Depth First Search)

Used to follow one possible path deeply before trying others.

Example:

$$Zone\ A \rightarrow Zone\ B \rightarrow Zone\ D$$

This helps test one possible fire spread direction.

3. Simple Path Cost

Each path has a simple difficulty value based on distance and danger.

$$Path\ Cost = Distance + Danger$$

Example: If:

- Distance = 5
- Danger = 3 (due to fire or smoke)

$$Cost = 8$$

The system prefers paths with lower cost.

4. Fire-Aware Path Update

If fire increases in a zone, that path becomes harder to use.

$$Updated\ Cost = Distance \times (1 + Fire\ Level)$$

Example: If:

- Distance = 10
- Fire Level = 0.5

$$Cost = 15$$

So that route is avoided if possible.

2.6 Hash-Based Indexing Layer

This layer is used to quickly store and retrieve forest data using a key-value system. Instead of searching all data one by one, the system uses a hash function to directly locate where the data is stored.

Example: If we want data for Zone 5, the system directly jumps to its storage location instead of checking all zones.

2.6.1 Structural Instances

- **H1: Primary Index Table** Stores main sensor and zone data using unique keys. Example:

$$Key = ZoneID \rightarrow Value = Temperature, Smoke, Humidity$$

- **H2: Collision Handling Table** Stores data when two keys produce the same location in the table. Example: Zone 3 and Zone 13 both map to same index, so one is stored separately.
- **H3: Fast Retrieval Cache** Stores frequently accessed data for quick access. Example: Zone 1 fire data is accessed repeatedly during emergency.

2.6.2 Functional Role

- H1 provides direct access to forest data using a key. Example: Enter Zone ID = 4 $\rightarrow$ instantly get its sensor readings.
- H2 handles conflicts when two different zones get the same storage position. Example: Zone 2 and Zone 12 both map to index 5, so one is stored in a separate list.
- H3 speeds up system performance by storing recently used data. Example: If Zone 3 data was accessed recently, it is retrieved faster next time.

2.6.3 Simple Hash Function Idea

The position of data in the table is calculated using:

$$Index = Key \mod TableSize$$

Example: If:

- Key = 7
- Table Size = 5

$$Index = 7 \mod 5 = 2$$

So data is stored at position 2.

2.7 System Monitoring Layer

This layer is responsible for continuously observing how the whole system is working during execution. It checks whether all modules (arrays, stacks, queues, trees, graphs) are running properly and identifies delays, overloads, or failures. The goal is to ensure the system does not slow down or break during forest simulation or emergency situations.

The monitoring layer collects performance data from every part of the system and converts it into measurable values such as time delay, processing load, and error frequency.

2.7.1 Functional Role

This layer tracks system health using simple measurable indicators. It does not store forest data, but instead observes how fast and efficiently the system processes that data.

- **Execution latency tracking** Measures how much time the system takes to process one operation or event. Example: If a fire alert takes 2 ms in normal condition but 10 ms during overload, the system detects slowdown.

  Basic formula:

  $$\text{Latency} = \text{Finish Time} - \text{Start Time}$$

- **Subsystem load analysis** Measures how much work each module (stack, queue, tree, graph) is handling at a given time. Example: If the queue system has 100 pending events while others have 10, it is overloaded.

  Load idea:

  $$\text{Load} = \frac{\text{Number of Active Tasks}}{\text{Processing Capacity}}$$

- **Bottleneck detection** Identifies the slowest part of the system that is delaying overall processing. Example: If graph routing is slow during fire spread calculation, it becomes a bottleneck and affects decision-making speed.

2.7.2 Metrics Explanation with Example

The system uses monitoring values to decide whether it is stable or overloaded:

- Normal condition: low latency, balanced load across all modules
- Warning condition: one module starts taking more time than others
- Emergency condition: system delay increases significantly and decisions slow down

Example scenario: During a forest fire simulation, the graph routing system recalculates shortest paths repeatedly. If this causes delay in queue processing, the monitoring layer detects imbalance and signals system optimization.

2.8 Final Principle

All structures operate under a unified principle:

> Each structural instance is an isolated computational actor participating in a synchronized multi-layer decision ecosystem.

3 System Design Philosophy

The IFAMDS system is a multi-layer computing system where all processing is done on well-structured environmental data. Each department works as an independent module with clear input and output rules. This means every part of the system takes data, processes it, and passes it forward in a fixed and predictable way. Because of this, data flows smoothly across all layers without confusion or randomness. All data is treated as structured computational objects instead of simple raw signals. This helps the system stay organized, consistent, and easy to manage even when multiple modules are working together at the same time.

3.1 Department 1: Environmental Data Acquisition and Structuring Layer

This department is the main input part of the IFAMDS system. It is responsible for collecting raw environmental data from distributed sensors and converting it into a consistent and usable system-wide representation.

The system operates on continuous streams of incoming signals generated from multiple sensor nodes deployed across different forest regions. Each incoming signal is first aligned according to its time of arrival so that all readings follow a proper sequence. Each reading is also linked with its physical location so the system knows exactly which forest region the data belongs to. After this, raw values are converted into a standard format so that all sensor outputs follow the same structure and can be processed uniformly.

Each sensor reading is processed through a structured validation stage before it is accepted into the system. This validation includes checking whether values lie inside physically possible limits, removing sudden incorrect spikes caused by sensor noise, and comparing the reading with nearby region values to ensure consistency across the forest grid. If a value

strongly differs from surrounding region values or previous recorded values, it is marked as unreliable and excluded from the main dataset.

The system also maintains organized internal representations of the environment so that both time-based changes and spatial distributions can be accessed efficiently. This means the system keeps track of how values change over time for each region, and also how values are arranged across all forest zones at the same moment.

These internal representations allow the system to support three main operations: storing continuous sensor updates in order, comparing new readings with previous environmental states, and retrieving region-based data for decision-making in later system layers.

3.2 Department 2: Event Memory and Temporal Reconstruction Layer

This department is responsible for storing all validated environmental data in a structured time-based form so that the system can track how forest conditions change over time. It acts as the memory component of IFAMDS and ensures that past, present, and updated environmental states remain accessible for later processing.

Each validated sensor reading is converted into a structured event record. Each event contains three main elements: the recorded value, the time of observation, and the region from which the data was collected. These event records are stored in a sequential order so that the system can follow the exact history of environmental changes step by step.

The system maintains a continuous chain of these event records so that new observations are always added after older ones. This allows the system to maintain a complete timeline of environmental behavior without losing previous states. Because of this structure, the system can move through data in chronological order whenever it needs to analyze how a situation developed.

In addition to forward storage, the system also supports revisiting previous event states when corrections are required. If a new reading contradicts earlier stored values, the system can trace back through earlier records to identify the point where inconsistency started. From that point, updated values can be applied to restore correctness in the stored history.

To ensure consistency, each new event is compared with both nearby time events and spatially related region data. If a newly stored event does not match expected behavior based on recent history or surrounding region values, it is flagged for review and the system attempts to reconstruct a corrected version using previously stored valid information.

This layer ensures that the system always maintains a reliable historical record of environmental changes, supports correction of past data when required, and enables reconstruction of missing or inconsistent information using stored event history.

3.3 Department 3: Execution Control and Computational Reasoning Layer

This department functions as the logical execution core of the IFAMDS system. It is responsible for processing environmental data and historical event records in a structured way to produce intermediate computational results that support system-wide decision processing.

The subsystem takes structured environmental datasets along with the verified event history and processes them step by step. It evaluates how the current environmental state changes over time by following the stored event sequences and applying controlled state transition logic.

Execution control is managed through an internal state tracking mechanism. This mechanism ensures that every step of computation is recorded so the system can continue processing without losing progress. It maintains execution continuity across multiple processing stages and ensures that intermediate results remain consistent throughout the computation process.

The system also includes a controlled rollback capability. This allows the execution process to return to the most recent valid state whenever required. Along with this, the system continuously monitors execution steps to detect any logical mismatch or inconsistency during processing.

When such inconsistencies are detected, the execution process is paused immediately. The system then restores the last verified stable state and restarts computation from that point. If needed, alternative processing paths are used to continue execution so that the system can complete reasoning without breaking overall system consistency.

3.4 Department 4: Task Scheduling and Priority Allocation Layer

This department defines the execution order of the system by organizing all computational tasks into priority-based categories.

Tasks are treated as separate computational units generated from environmental monitoring, event validation, and emergency response triggers. These tasks are arranged into priority levels based on urgency, timing requirements, and overall system impact.

The scheduling mechanism continuously adjusts execution order based on changing environmental conditions. Tasks that represent higher risk or higher importance are executed first, while normal monitoring tasks continue in the background without interruption.

The system also distributes workload across available processing resources. This prevents overload on a single component and ensures smooth execution even when a large number of tasks are generated at the same time.

3.5 Department 5: Hierarchical Decision Intelligence Layer

This department is responsible for generating system-level decisions based on structured environmental information.

Decision-making is performed at multiple levels, including local zone-level decisions, regional coordination decisions, and full system-wide global decisions.

Each decision is evaluated using current environmental data, previously validated historical records, and predicted future behavior of the system. If multiple decision options conflict with each other, the system resolves them using a hierarchical selection process that ensures only the most stable and suitable decision is finalized.

3.6 Department 6: Spatial Connectivity and Graph-Based Routing Layer

This department represents the forest environment as a graph structure where each region is treated as a node and connections between regions are treated as edges.

The system continuously updates these connections based on changing environmental conditions such as fire spread behavior, terrain difficulty, and movement constraints.

Path calculations are updated dynamically when conditions change, allowing the system to adjust movement routes and propagation paths in real time.

3.7 Department 7: Indexing and Retrieval Acceleration Layer

This department is responsible for fast retrieval of environmental and system data.

It uses key-value based storage to map sensor IDs, event records, and environmental states for quick access. It also uses caching to store frequently accessed data so that repeated queries can be answered faster.

This ensures that important data can be retrieved in constant time without scanning the entire dataset, reducing delay in decision-making and system processing.

3.8 Department 8: System Monitoring and Adaptive Optimization Layer

This department continuously monitors the performance of the entire system.

It tracks important metrics such as processing delay, workload distribution, and frequency of unusual system behavior. Based on this information, the system adjusts its internal configuration.

These adjustments include redistributing workload, changing execution priorities, and tuning processing limits. This ensures the system remains stable and responsive under both normal and high-load conditions while maintaining consistent and reliable operation.

4 Scenario-Based System Evaluation

4.1 Scenario 1: Cascading Fire and Resource Conflict Resolution System

A fire starts in Zone 3 and begins spreading toward Zone 4 and Zone 6. Sensor data such as temperature, smoke level, and wind is continuously collected and added to the system to represent the current forest condition.

Each new reading is checked to make sure it is correct and consistent with nearby zones and previous values. If any reading looks wrong or unstable, it is removed from active use so it does not affect the system.

All valid readings are stored in the order they arrive so the system can track how the situation changes over time. If any mistake or instability is detected in the current state, the system goes back to the last correct state before continuing updates.

When the situation becomes dangerous, important updates are handled first while normal monitoring is delayed for a short time. A safe copy of the current state is also saved so the system can recover if needed.

The system estimates fire strength by checking temperature, smoke, and how close the affected zones are to each other. Based on this, attention is focused on Zone 3, response in Zone 4 is controlled, and Zone 6 continues to be monitored.

All updates and changes are saved so the system can always track what happened and keep the information consistent.

4.2 Scenario 2: Sensor Failure and System Reconstruction

A problem occurs in Zone 2 where several sensors start sending incomplete or incorrect readings. Because of this, gaps appear in the system's view of the forest, and it becomes difficult to form a correct overall picture of the environment. Some readings are rejected because they are missing values or do not look reliable.

To keep the system running properly, the system looks back at previously correct information and uses it in the order it was recorded. It finds the last stable state of the environment and uses it as a starting point for recovery.

From this point, missing values are estimated using past patterns and information from nearby zones. This helps rebuild the missing part of the system step by step.

If the rebuilt version still creates errors or inconsistencies, the system goes back again to an earlier correct state and tries a different way of rebuilding the data. This process continues until the system becomes stable again.

Once a correct and complete view is formed, the restored information is added back into the system so that Zone 2 can continue normal operation under proper checking.

4.3 Scenario 3: Multi-Factor Anomaly Escalation System

At the same time, different unusual events start appearing in the forest. In one area, animals move in unexpected patterns. In another, fire risk increases. In a separate zone, human movement is detected.

All these signals are combined into one shared view of the forest so the system can understand the overall situation instead of treating each event separately.

The system compares current behavior with past patterns and checks whether multiple conditions together are increasing danger.

If risk becomes higher in some zones, those areas are marked for urgent attention, while others continue normal observation.

The system also checks how one event may affect nearby zones so it can respond early before the situation spreads.

4.4 Scenario 4: System Overload and Load Redistribution

The system receives a very large number of incoming updates at the same time, which starts slowing down overall processing. Because of this, new information cannot be handled at normal speed, and delays begin to appear in the system response.

To handle this situation, the system first separates important updates from less important ones. Critical information is processed immediately, while lower priority updates are temporarily paused.

The system keeps a safe and stable version of the current forest state so that no incorrect or partial updates affect the main system view during this high-load period.

After that, the processing order is adjusted so that the workload is balanced more evenly across available resources. Frequently needed information is kept ready for quick access to avoid repeating the same calculations again.

Once the load decreases and the system becomes stable again, the paused updates are slowly processed and added back into the system in a controlled way.

Finally, the system returns to normal operation with stable performance and consistent environmental tracking.

4.5 Scenario 5: Global Multi-Zone Emergency Synchronization

A large-scale emergency occurs across multiple zones, where simultaneous events generate a global environmental representation. Due to asynchronous updates, inconsistencies arise between regional states, preventing a unified system view.

The system revisits previously validated observations to identify the most recent globally consistent state. Conflicting data is isolated, and only consistent segments are retained for integration.

If inconsistencies persist, a correction state is constructed by combining verified historical observations with current validated inputs. Regional conflicts are resolved through higher-level coordination, ensuring consistent decision-making across zones.

Once a coherent global state is achieved, the system generates a synchronized response strategy with balanced resource allocation. Continuous validation maintains alignment across all regions, ensuring stability and consistency.

5 System Deliverables and Implementation Requirements

The Intelligent Forest Advisory & Multi-Structure Decision System (IFAMDS) must be implemented as a **C++ console-based menu-driven simulation system**. The interface is terminal-based, and all functionalities are demonstrated through interactive menu navigation.

This structure is provided as a reference only. Students may modify the menu design as long as all required modules and scenarios are implemented.

6 System Menu (Main Menu Structure)

The IFAMDS system operates through a single unified menu-driven interface. All system operations are accessed through this hierarchical control menu.

6.1 Main Menu

- 1. Input Environmental Data
  - 1.1 Add Sensor Reading (Temperature, Smoke, Wind)

  - 1.2 Store Data in Dynamic Array
  - 1.3 Compare with Static Baseline
  - 1.4 Validate and Filter Noise
- 2. View Forest Grid Status
  - 2.1 Display 1D Time Series Data
  - 2.2 Display 2D Forest Zone Matrix
  - 2.3 View Zone-wise Conditions
- 3. Event Memory System
  - 3.1 Store Event (Linked List)
  - 3.2 Traverse Event History Forward
  - 3.3 Traverse Event History Backward
  - 3.4 Circular Event Monitoring
  - 3.5 Restore Last Stable State
- 4. Fire Detection and Control
  - 4.1 Detect Fire Risk (Threshold Check)
  - 4.2 Trigger Emergency Alert
  - 4.3 Priority-Based Fire Response
  - 4.4 Simulate Fire Spread (Graph BFS)
  - 4.5 Allocate Firefighting Resources
- 5. Task Scheduling System
  - 5.1 Add Routine Task (Queue)
  - 5.2 Add Surveillance Task
  - 5.3 Add Emergency Task (Priority Queue)
  - 5.4 Process Tasks (FIFO / Priority)
  - 5.5 Pause and Resume Tasks
- 6. Decision System
  - 6.1 Compute Risk Score

  - 6.2 Zone-Level Decision Tree
  - 6.3 Regional Decision Processing
  - 6.4 Global Emergency Decision
  - 6.5 Execute Final Action
- 7. Spatial Routing System
  - 7.1 Load Graph (Adjacency List)
  - 7.2 Load Graph (Adjacency Matrix)
  - 7.3 BFS Traversal (Fire Spread)
  - 7.4 DFS Traversal (Deep Analysis)
  - 7.5 Compute Safe Path
  - 7.6 Update Blocked Routes
- 8. Hash-Based Fast Access System
  - 8.1 Insert Data (Hash Table)
  - 8.2 Retrieve Data (O(1) Access)
  - 8.3 Handle Collisions
  - 8.4 Update Cache
  - 8.5 View Index Table
- 9. System Monitoring
  - 9.1 Monitor System Load
  - 9.2 Track Execution Time
  - 9.3 Detect Bottlenecks
  - 9.4 Optimize Performance
  - 9.5 View System Health
- 10. Scenario Simulation
  - 10.1 Cascading Fire Scenario
  - 10.2 Sensor Failure Scenario
  - 10.3 Multi-Factor Anomaly Scenario
  - 10.4 System Overload Scenario

  - 10.5 Global Emergency Scenario
  - 10.6 Run Full System Simulation

- 0. Exit System